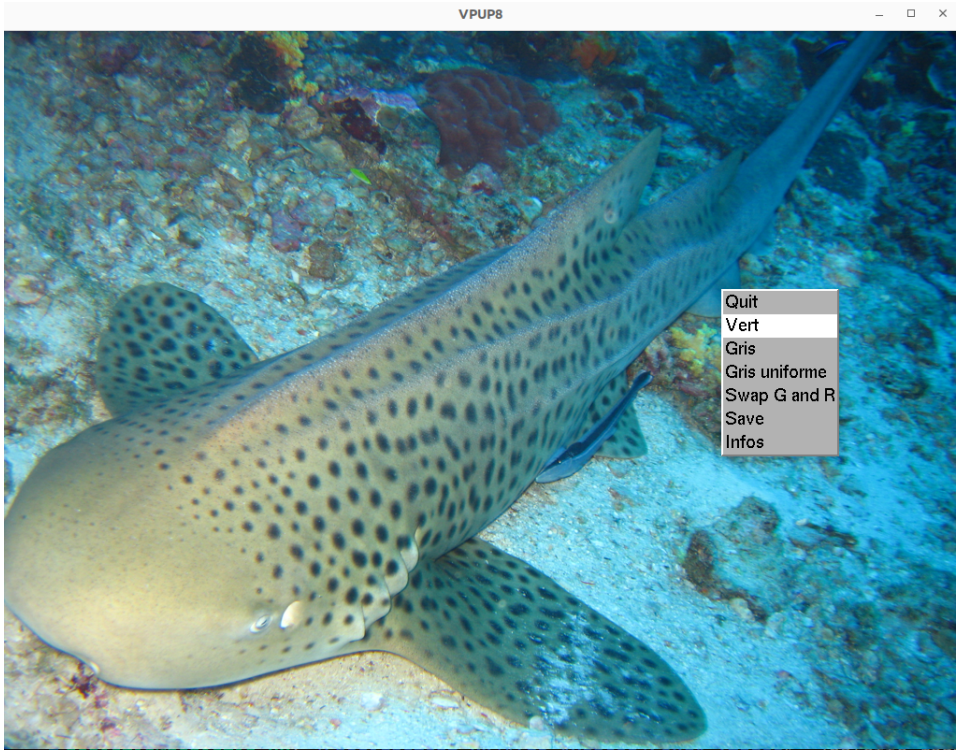
2023-2024 Université Paris 8
Algorithmique avancée Licence 3 Informatique

TP n°4 – Images : modification et compression sans perte.



Vous trouverez dans l'archive *Images.zip* sur Moodle plusieurs fichiers C dont :

- *ima.h*, le header, qui contient la structure d'image, les prototypes et les directives de préprocesseur permettant l'inclusion des bibliothèques graphiques notamment.
- *ppm.c* qui contient des fonctions de gestion d'images PPM,
- *modif.c*, qui contient des fonctions de modification d'image PPM.
- *main.c* qui contient des fonctions de gestion de la fenêtre graphique et du menu (voir l'image ci-dessus).
- un *makefile* pour compiler tout ça, qu'il faudra peut-être réadapter à votre configuration. L'exécution sera alors réalisée par

```
./palette Chemin_vers_image
```

0 Initialisation

Récupérer les fichiers de l'archive *Images.zip*, et vérifiez que vous pouvez lire et afficher des images. Sinon, il vous faudra peut-être modifier l'inclusion des librairies dans *ima.h* ou installer les librairies nécessaires si vous utilisez votre machine.

1 Modification d'images

En vous inspirant de la fonction *vert* qui modifie une image en mettant dans chaque pixel une teinte de vert à 255 et une teinte de rouge et de bleu à 0, écrire des fonctions :

1. *vert_moy* qui met dans chaque pixel une teinte de vert qui correspond à la moyenne des teintes de rouge, vert et bleu du pixel.

2. *vert_uniform* qui met dans chaque pixel une teinte de vert qui correspond à la moyenne pondérée des teintes de rouge (poids 2), vert (poids 4) et bleu (poids 1) du pixel.

3. *rouge_origin* qui met dans chaque pixel une teinte de rouge qui est la même que celle de l'image d'origine, et met toutes les teintes de vert et de bleu à 0.

4. *gris_uniform* qui met dans chaque pixel une teinte de gris qui est la moyenne pondérée des teintes de rouge (poids 2), vert (poids 4) et bleu (poids 1) du pixel. On rappelle que pour obtenir du gris, on mettra la même valeur dans R,G et B.

5. *swap_green_red*, qui échange dans chaque pixel les teintes de rouge et de vert, et qui met toutes les teintes de bleu à 0.

2 Algorithme de compression sans perte

L'objectif de cette partie est d'implémenter l'algorithme de compression sans perte *Run Length Encoding* (RLE).

1. Écrire une fonction qui permet de trier les 3 couleurs d'une image : on stockera tous les *GLubyte* de rouge, de vert et de bleu dans un grand tableau.

2. Écrire une fonction qui permet de compresser un tableau de *GLubyte* selon le principe de l'algorithme RLE naïf.

3. Écrire une fonction qui décompresse un tableau de *GLubyte*, puis une fonction qui reconstitue l'image à partir du tableau trié de couleurs.

4. Écrire une version améliorée de cet algorithme en ajoutant dans la compression l'amélioration vue en cours avec les multiplicateurs négatifs.